Universidad de San Carlos de Guatemala
Facultad de Ingeniería · Escuela de Ciencias y Sistemas
Organización Computacional

Práctica 2 — LogicCalc

Lógica Binaria y Combinacional

Unidad Aritmética Lógica (ALU)

Integrante 1	Alison Melysa Pérez Blanco Carné: 202400023
Integrante 2	José Fernando Ramírez Ambrocio Carné: 202400195
Integrante 3	Jackeline Stephany Rivera Argueta Carné: 202401685
Integrante 4	Emily Maritza Tepeu Guacamaya Carné: 202402955
Grupo	10
Sección	C
Auxiliares	Rony Omar Miguel López / Melvin Alexander Valencia Estrada
Catedrático	Ing. Fernando José Paz
Fecha de entrega	21/03/2026

1. Introducción

La presente práctica, denominada LogicCalc, consiste en el diseño, simulación y construcción física de una Unidad Aritmética Lógica (ALU) básica fundamentada en lógica combinacional pura. El proyecto fue desarrollado en el marco del curso de Organización Computacional de la Facultad de Ingeniería de la Universidad de San Carlos de Guatemala, como ejercicio integrador de los conceptos de álgebra de Boole, circuitos combinacionales y arquitectura de sistemas digitales.

Una ALU es el componente central de cualquier procesador: es el módulo encargado de ejecutar todas las operaciones aritméticas y lógicas que le ordena la unidad de control. Comprender su funcionamiento desde las compuertas lógicas más básicas permite al estudiante apreciar de forma concreta cómo una computadora realiza internamente desde una simple suma hasta operaciones de comparación o lógica bit a bit.

El sistema LogicCalc implementado recibe dos operandos de 4 bits (A y B) y, mediante una señal de control de 3 bits (C, B, A del controlador), selecciona y ejecuta una de ocho operaciones posibles: suma, resta, multiplicación, potencia, AND, OR, NAND y XNOR. Los resultados aritméticos y comparativos se visualizan en displays de 7 segmentos, mientras que los resultados lógicos se presentan en LEDs de 4 bits.

Para la implementación se utilizaron únicamente componentes TTL autorizados: compuertas 7408 (AND), 7432 (OR), 7486 (XOR), 7404 (NOT), sumadores 7483, comparador 7485, multiplexores 74157 y 74153, y el decodificador 7448 para los displays. El circuito fue diseñado sin elementos secuenciales, garantizando que cada salida depende exclusivamente de las entradas presentes en ese instante.

2. Objetivos

2.1 Objetivo General

Diseñar, simular y construir físicamente una calculadora digital denominada LogicCalc, basada exclusivamente en lógica combinacional TTL, capaz de ejecutar operaciones aritméticas (suma, resta, multiplicación y potencia) y lógicas (AND, OR, NAND, XNOR) sobre operandos de 4 bits, mostrando los resultados en displays de 7 segmentos y LEDs.

2.2 Objetivos Específicos

- Implementar una unidad aritmética que realice suma y resta binaria usando únicamente compuertas lógicas (sin sumadores integrados para esas operaciones), aplicando complemento a dos para la resta.
- Diseñar la multiplicación binaria de 4 bits utilizando compuertas AND como generadoras de productos parciales y sumadores 7483 para la acumulación de resultados.
- Construir el módulo de potencia (A^2 y A^3) empleando compuertas NOT, XOR y AND para derivar los bits del resultado.
- Implementar las cuatro operaciones lógicas (AND, OR, NAND, XNOR) bit a bit sobre los operandos A y B usando exclusivamente las compuertas TTL permitidas.
- Diseñar la unidad comparativa con el integrado 7485, multiplexores 74157 para seleccionar y enrutar el mayor y el menor, y el decoder 7448 para la visualización en displays de 7 segmentos.
- Integrar un controlador basado en multiplexores 74153/74157 que seleccione qué módulo está activo según la señal de control de 3 bits.
- Validar el funcionamiento completo mediante simulación en Proteus y pruebas sobre el circuito físico en protoboard y PCB.

3. Funciones Booleanas

Las siguientes funciones booleanas describen la lógica de cada módulo del sistema LogicCalc. Los operandos son $A = A_3A_2A_1A_0$ y $B = B_3B_2B_1B_0$ (4 bits cada uno). Todas las funciones son combinacionales.

3.1 Unidad Aritmética — Sumador (con compuertas)

La suma binaria A + B se implementa con sumadores completos (Full Adders) construidos con compuertas XOR (7486), AND (7408) y OR (7432). Cada bit de suma S_i y el acarreo de salida C(i+1) se obtienen con:

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C(i+1) = (A_i \cdot B_i) + (C_i \cdot (A_i \oplus B_i))$$

Para los 4 bits, expandiendo con $C_0 = 0$ (sin acarreo inicial):

$$S_0 = A_0 \oplus B_0$$

$$C_1 = A_0 \cdot B_0$$

$$S_1 = A_1 \oplus B_1 \oplus C_1 = A_1 \oplus B_1 \oplus (A_0 \cdot B_0)$$

$$C_2 = (A_1 \cdot B_1) + (C_1 \cdot (A_1 \oplus B_1))$$

$$S_2 = A_2 \oplus B_2 \oplus C_2$$

$$C_3 = (A_2 \cdot B_2) + (C_2 \cdot (A_2 \oplus B_2))$$

$$S_3 = A_3 \oplus B_3 \oplus C_3$$

$$Cout = (A_3 \cdot B_3) + (C_3 \cdot (A_3 \oplus B_3)) \quad \leftarrow \text{acarreo de salida (5to bit)}$$

Resultado final = Cout S_3 S_2 S_1 S_0 (5 bits). Se convierte a BCD para los displays mediante la lógica de corrección del 7483.

3.2 Unidad Aritmética — Restador (Complemento a 2)

La resta A − B se implementa convirtiendo B a su complemento a dos y sumándolo a A. El complemento a dos de B se obtiene negando cada bit con 7404 (NOT) y añadiendo 1 al acarreo inicial:

$$B'_i = Bi' \quad \text{(NOT de cada bit, usando 7404)}$$

$$A - B = A + B' + 1 \quad (C_0 = 1 \text{ en la cadena de sumadores})$$

Las funciones booleanas del restador son idénticas al sumador, pero con B' en lugar de B y $C_0 = 1$:

$$R0 = A_0 \oplus B0' \oplus 1 = A_0 \oplus B0'' \quad \text{(con acarreo inicial = 1)}$$

$$R1 = A_1 \oplus B1' \oplus C_1$$

$$R2 = A_2 \oplus B2' \oplus C_2$$

$$R3 = A_3 \oplus B3' \oplus C_3$$

Validación (A ≥ B): si el acarreo de salida Cout = 1, el resultado es positivo (A ≥ B). Si Cout = 0, entonces A < B y se muestra "EE" en el display (se detecta con la lógica del controlador).

3.3 Unidad Aritmética — Multiplicador (AND + Sumadores 7483)

La multiplicación binaria A × B de 4 bits × 4 bits genera un producto de hasta 8 bits. Se implementa generando los productos parciales con compuertas AND (cada bit de A con cada bit de B) y acumulándolos con sumadores 7483:

Productos parciales ($PP_{ij} = A_i \cdot B_j$):

```
PP0: P00=A0B0   P10=A1B0   P20=A2B0   P30=A3B0
PP1: P01=A0B1   P11=A1B1   P21=A2B1   P31=A3B1
PP2: P02=A0B2   P12=A1B2   P22=A2B2   P32=A3B2
PP3: P03=A0B3   P13=A1B3   P23=A2B3   P33=A3B3
```

Bit menos significativo del resultado: $P_0 = A_0 \cdot B_0$ (directo, sin suma).

Los bits P_1 a P_7 se obtienen sumando los productos parciales desplazados con dos etapas de sumadores 7483, siguiendo el algoritmo de multiplicación binaria estándar.

3.4 Unidad Aritmética — Potencia (NOT, XOR, AND)

La operación de potencia calcula A^2 (si B=0010) o A^3 (si B=0011). Para $A^2 = A \times A$ se aplican las mismas identidades que en el multiplicador pero con $A_i = B_i$. Las funciones booleanas se simplifican porque los productos de un número consigo mismo tienen propiedades especiales:

Para A^2 ($A_i \cdot A_i = A_i$), los productos cruzados se reducen:

$$P_0 = A_0 \cdot A_0 = A_0$$

$$P_1 = 2 \cdot (A_1 \cdot A_0) \quad \rightarrow \text{suma de acarreos}$$

$$P_2 = A_1 \cdot A_1 + 2 \cdot (A_2 \cdot A_0) = A_1 + 2 \cdot (A_2 \cdot A_0)$$

La lógica de potencia utiliza compuertas NOT (7404) para invertir bits cuando se requiere el complemento, XOR (7486) para detectar diferencias entre bits y AND (7408) para los productos parciales, alimentando finalmente al sumador 7483 para obtener el resultado.

3.5 Unidad Lógica (compuertas puras)

Todas las operaciones lógicas se aplican bit a bit sobre los 4 bits de A y B. Se usan exclusivamente compuertas TTL:

3.5.1 AND (integrado 7408)

$$F_AND_i = A_i \cdot B_i \qquad \text{para } i = 0,1,2,3$$

3.5.2 OR (integrado 7432)

$$F_OR_i = A_i + B_i \qquad \text{para } i = 0,1,2,3$$

3.5.3 NAND (7408 + 7404)

$$F_NAND_i = (Ai \cdot Bi)' \qquad \text{para } i = 0,1,2,3$$

Se implementa conectando la salida de la compuerta AND al inversor 7404.

3.5.4 XNOR (7486 + 7404)

$$F_XNOR_i = (Ai \oplus Bi)' \qquad \text{para } i = 0,1,2,3$$

Se implementa conectando la salida del XOR (7486) al inversor 7404.

La salida de la unidad lógica son 4 bits que se muestran directamente en 4 LEDs. Las unidades aritmética y lógica son mutuamente excluyentes, controladas por el bit C del controlador (C=0: aritmética; C=1: lógica).

3.6 Unidad Comparativa (7485 + 74157 + 7448)

El comparador 7485 recibe los operandos A y B de 4 bits y genera tres señales de salida:

```
A_MAYOR_B  (A > B)  →  salida pin 5 del 7485
A_IGUAL_B  (A = B)  →  salida pin 6 del 7485
A_MENOR_B  (A < B)  →  salida pin 7 del 7485
```

Los multiplexores 74157 usan estas señales para seleccionar qué valor (A o B) se envía a cada display:

```
Display MAYOR:  Si A_MAYOR_B=1 → muestra A;  Si A_MENOR_B=1 → muestra
B;  Si A_IGUAL_B=1 → muestra A (=B)
Display MENOR:  Si A_MAYOR_B=1 → muestra B;  Si A_MENOR_B=1 → muestra
A;  Si A_IGUAL_B=1 → muestra A (=B)
```

El decoder 7448 convierte el valor BCD de 4 bits seleccionado a los 7 segmentos del display:

```
a = f(D,C,B,A)  →  implementada internamente por el 7448
b = f(D,C,B,A)  →  id.
...
g = f(D,C,B,A)  →  id.
```

Solo se evalúan valores del 0 al 9 en la unidad comparativa.

4. Mapas de Karnaugh

Se presentan los mapas de Karnaugh para las funciones de salida de los módulos críticos. Las variables AB corresponden a los bits de mayor peso y CD a los de menor peso del operando o función evaluada. Las celdas en verde representan mintérminos = 1.

Nota: Para la suma y la resta los mapas se presentan por bit de salida; para la unidad lógica y comparativa se muestran las funciones de control del multiplexor y del decoder.

4.1 Sumador — Bit S_0 (función: $A_0 \oplus B_0$)

Variables: A_0 (A), B_0 (B). Mapa 2×2 simplificado a 4-variable con C=D=0:

Mapa K — S_0 ($A_0 \oplus B_0$) | variables: $A_1A_0 \setminus B_1B_0$

AB \ CD	00	01	11	10
00	0	1	0	1
01	1	0	1	0
11	0	1	0	1
10	1	0	1	0

SOP simplificada: $S_0 = A_0 \oplus B_0$ (no simplifica; el XOR es la forma mínima)

4.2 Sumador — Acarreo C_1 (función: $A_0 \cdot B_0$)

Mapa K — C_1 ($A_0 \cdot B_0$) | variables: $A_1A_0 \setminus B_1B_0$

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	0	1
11	0	0	0	0
10	0	1	0	1

SOP simplificada: $C_1 = A_0 \cdot B_0$ (grupo de 2 celdas; función AND mínima)

4.3 Restador — Bit R0 (función: $A_0 \oplus B0'$, con C_0=1)

Mapa K — R0 | variables: $A_1A_0 \setminus B_1B_0$

AB \ CD	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

SOP simplificada: $R0 = A_0 \oplus B0' = A_0$ XNOR B_0 (no simplifica más; forma mínima con XOR+NOT)

4.4 Unidad Lógica — NAND (función: (Ai·Bi)', por bit)

Mapa K — F_NAND | variables: AB \ (para un bit i)

AB \ CD	00	01	11	10
00	1	1	1	0
01	1	1	1	0
11	1	1	1	0
10	1	1	1	0

SOP simplificada: F_NAND = (A·B)' = A' + B' (Ley de De Morgan; 3 grupos de 4)

4.5 Unidad Lógica — XNOR (función: (Ai⊕Bi)', por bit)

Mapa K — F_XNOR | variables: AB \ (para un bit i)

AB \ CD	00	01	11	10
00	1	0	1	0
01	0	1	0	1
11	1	0	1	0
10	0	1	0	1

SOP simplificada: F_XNOR = A'·B' + A·B (dos grupos de 4 en diagonal; equivale a NOT XOR)

4.6 Comparador — Señal A_MAYOR_B (A > B) para 2 bits (A_1A_0 vs B_1B_0)

El 7485 implementa internamente la comparación. Para ilustrar el principio, se muestra el mapa para 2 bits:

Mapa K — A_MAYOR_B(A>B) | variables: A_1A_0 \ B_1B_0

AB \ CD	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	1	0	0	0
10	1	1	1	0

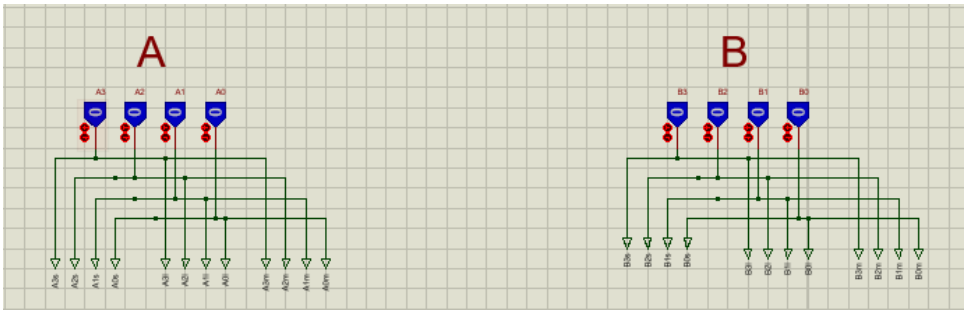
SOP simplificada: A_MAYOR_B = A_1·B1' + A_1·A_0·B0' + A_0·B1'·B0' (simplificación con 3 grupos)

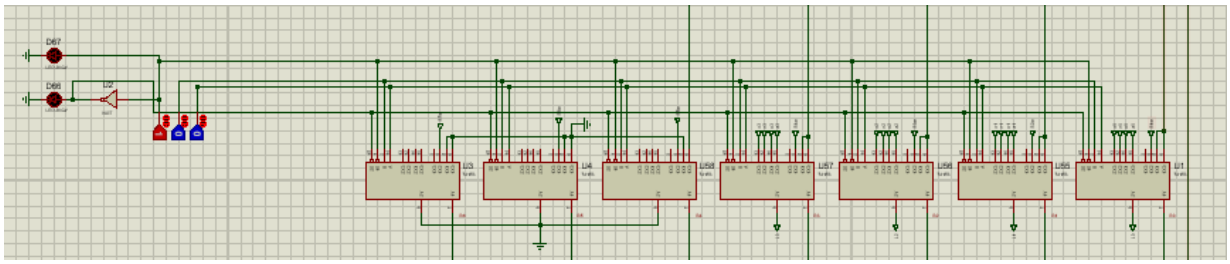
5. Simulación en Proteus

A continuación, se presentan las capturas del circuito LogicCalc implementado en Proteus. Se muestra primero una vista general y luego cada módulo por separado.

5.1 Controlador

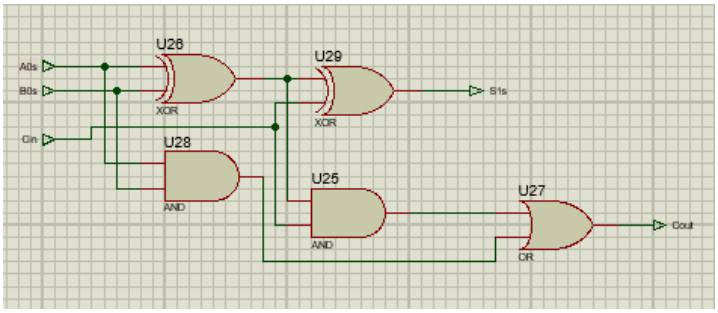
El controlador recibe la señal de 3 bits CBA y usa multiplexores 74153 para activar el modulo correcto y enrutar su salida. Con C = 0 se habilita la unidad aritmética (bits B y A seleccionan cual de las cuatro operaciones: 00 = suma, 01 = resta, 10 = multiplicacion, 11 = potencia). Con C = 1 se habilita la unidad logica (00 = AND, 01 = OR, 10 = NAND, 11 = XNOR). La unidad comparativa opera de forma independiente y siempre esta activa.

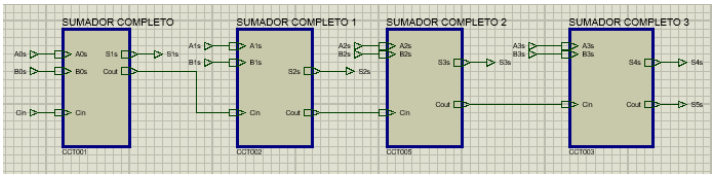


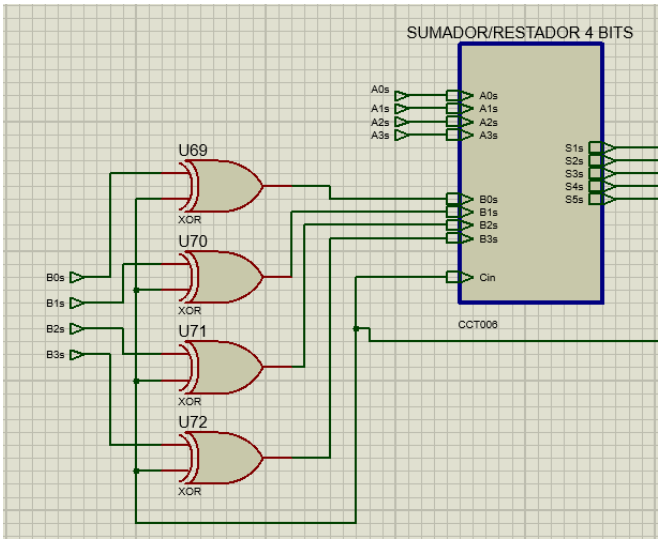


5.2 Sumador

El sumador construye un sumador completo (Full Adder) de 4 bits usando únicamente compuertas 7486 (XOR) para los bits de suma, 7408 (AND) y 7432 (OR) para el cálculo del acarreo. El acarreo se propaga desde el bit menos significativo (S0) hasta el más significativo (S3), con un quinto bit de acarreo de salida que permite representar resultados mayores a 15.

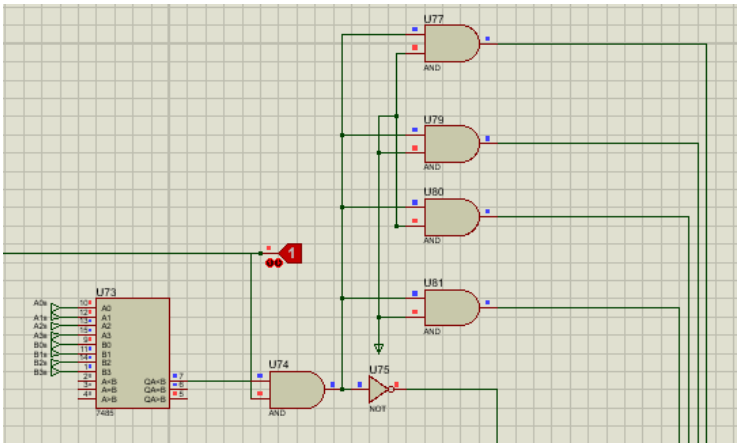


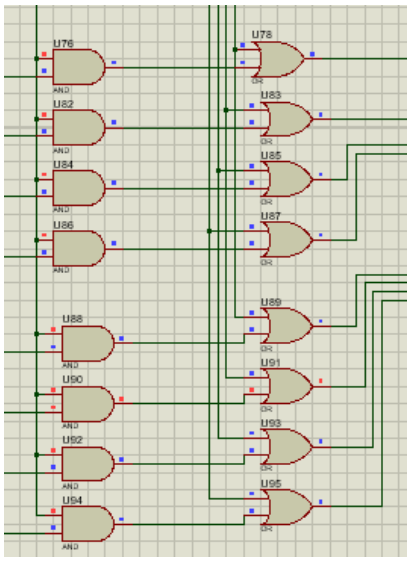




5.3 Restador

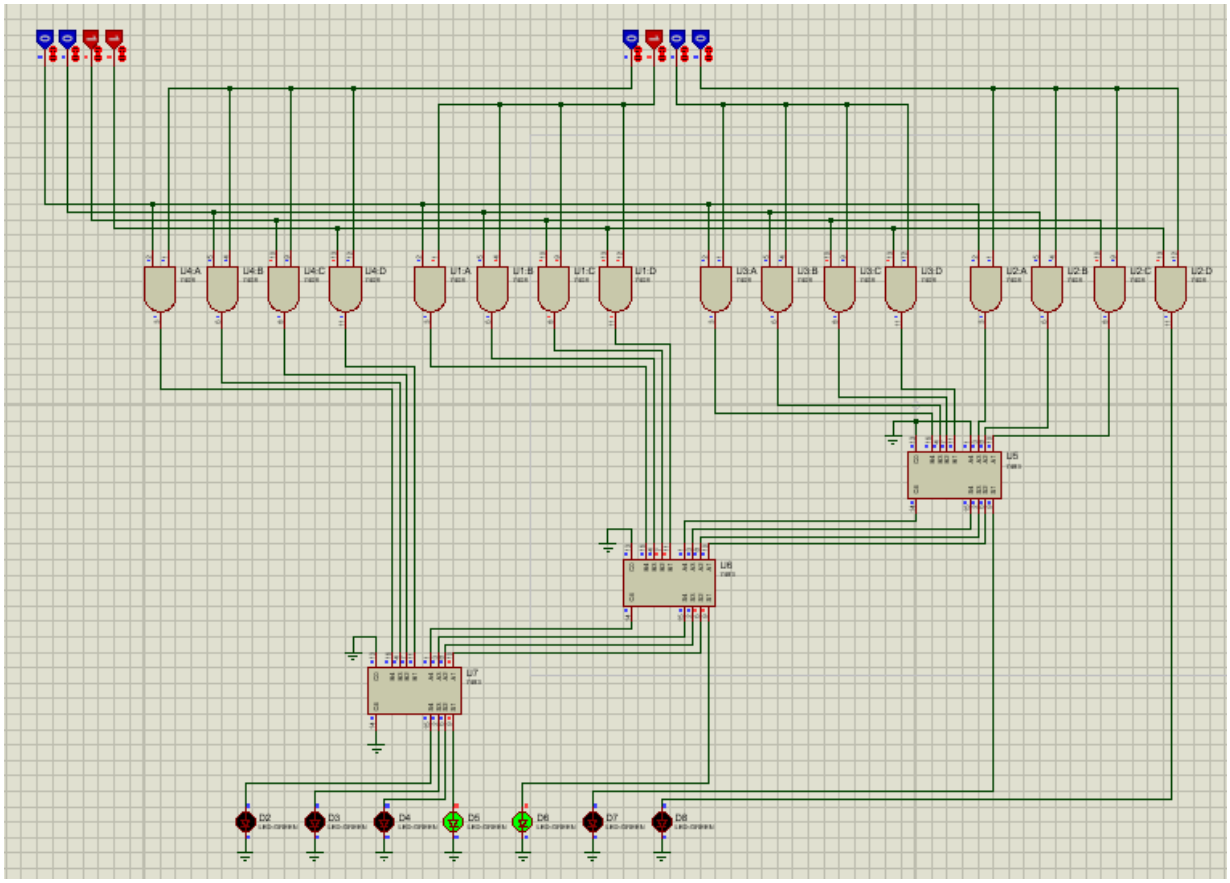
La resta A - B se realiza convirtiendo B a su complemento a 2: primero se invierten todos los bits de B con compuertas 7404 (NOT), y luego se fuerza el acarreo inicial C0 = 1 en la cadena de sumadores, lo que equivale a sumar 1. Si el acarreo de salida es 0 significa que A < B y el resultado no es representable, por lo que se muestra EE en el display.





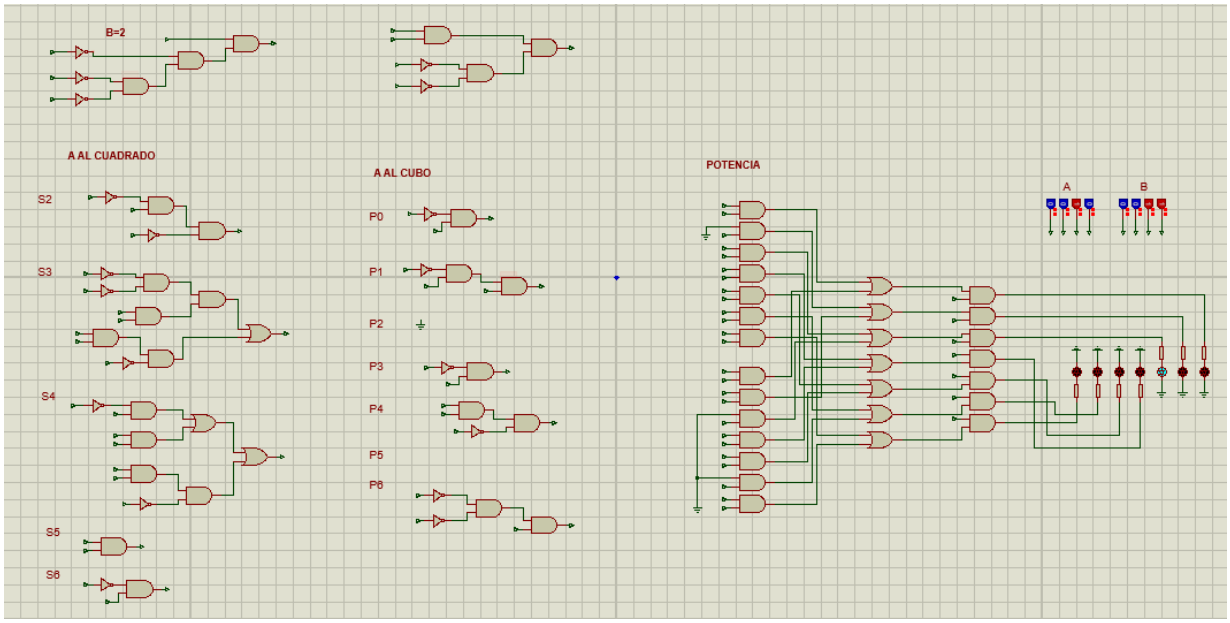
5.4 Multiplicador

El multiplicador genera los 16 productos parciales (cada bit de A con cada bit de B) usando compuertas 7408 (AND). Estos productos parciales se desplazan y acumulan en dos etapas de sumadores 7483 en cascada, siguiendo el algoritmo clasico de multiplicacion binaria. El bit menos significativo del resultado (P0) es directo: P0 = A0 · B0, sin necesidad de suma.



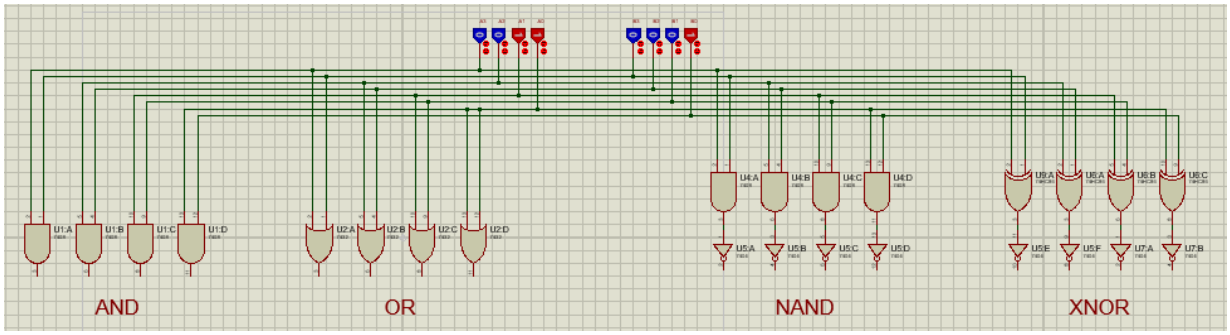
5.5 Potencia

El modulo de potencia tiene dos subcircuitos independientes construidos con compuertas NOT (7404) y AND (7408): uno calcula A al cuadrado (salidas S2-S6) y otro calcula A al cubo (salidas P0-P6). Ambos generan sus bits de resultado de forma combinacional pura, sin usar sumadores integrados. Un tercer bloque a la derecha actua como selector: recibe todas las salidas de ambos subcircuitos y, segun si B=2 o B=3, habilita con compuertas AND cuales bits pasan al resultado final. Los bits seleccionados pasan luego al conversor BCD antes de llegar a los displays.



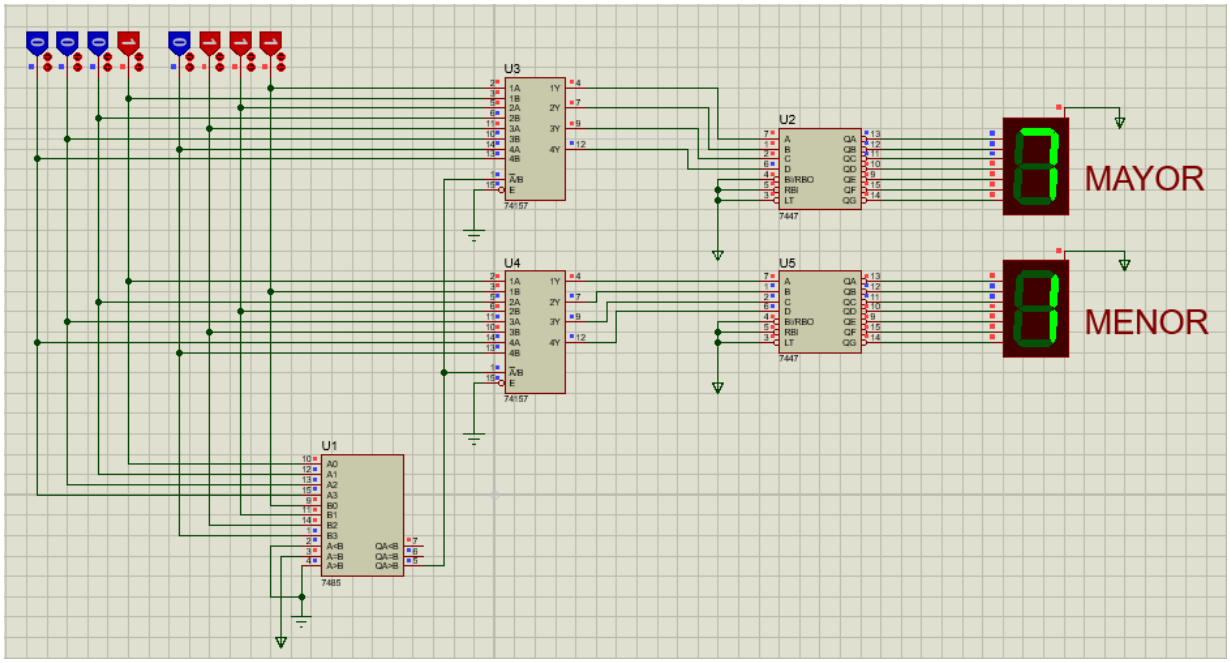
5.6 Unidad logica

Las cuatro operaciones lógicas se aplican bit a bit sobre los 4 bits de A y B. AND usa directamente el 7408; OR usa el 7432; NAND conecta la salida del 7408 a un inversor 7404; XNOR conecta la salida del 7486 a un inversor 7404. Los cuatro bits del resultado se muestran en LEDs individuales. Esta unidad es mutuamente excluyente con la aritmética: el bit C del controlador determina cual esta activa.



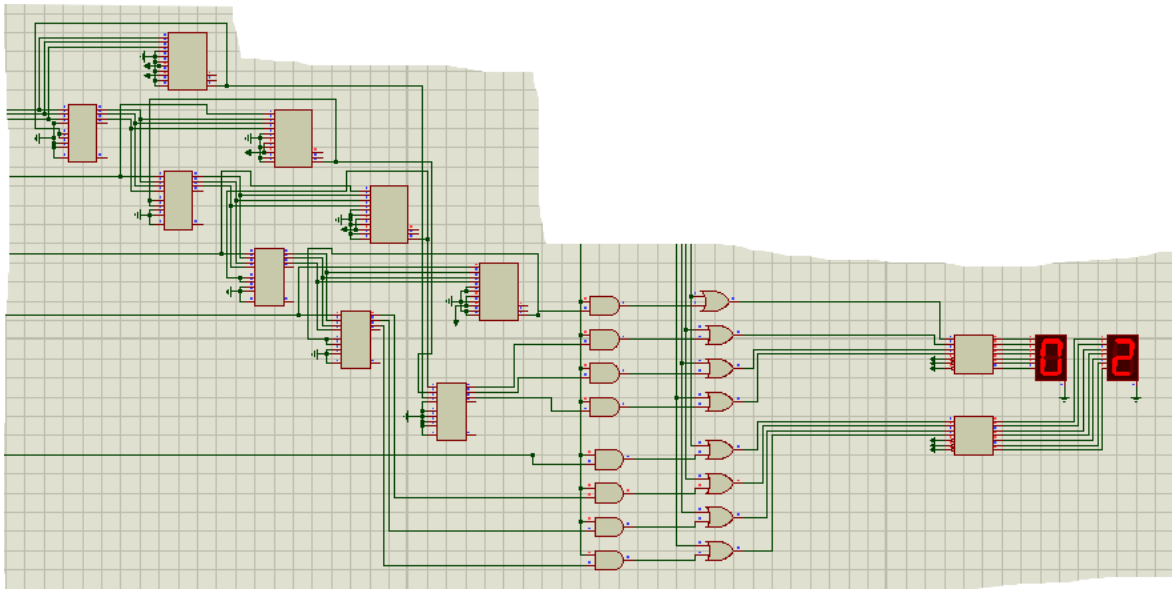
5.7 Unidad comparativa

El comparador 7485 recibe A y B de 4 bits y activa una de tres salidas: A_MAYOR_B, A_IGUAL_B o A_MENOR_B. Los multiplexores 74157 usan estas señales para seleccionar dinamicamente que valor (A o B) se envia a cada display: el mayor al display izquierdo y el menor al derecho. El decoder 7448 convierte cada valor BCD a los 7 segmentos del display correspondiente.



5.8 Conversor BCD y displays

El resultado binario de la unidad aritmética puede superar el valor 9, por lo que no puede enviarse directamente al decoder 7448. El conversor BCD recibe los bits del resultado y los separa en decenas y unidades, generando dos nibbles BCD de 4 bits cada uno. Cada nibble alimenta un decoder 7448 independiente, que lo convierte en las 7 señales de segmento (a-g) necesarias para encender correctamente cada display. Asi se logra mostrar resultados de dos cifras (00 a 99) en los dos displays de 7 segmentos.



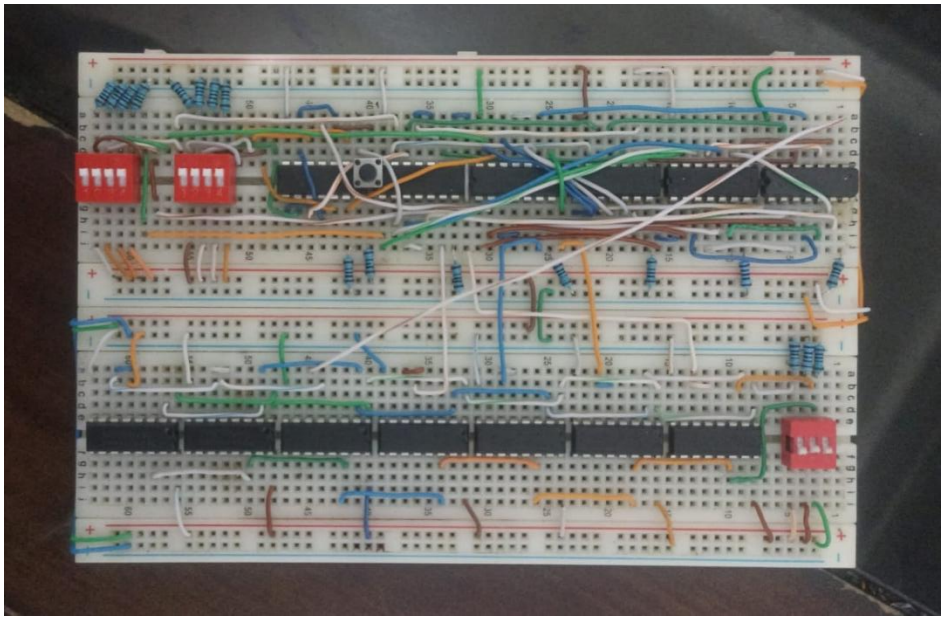
6. Circuito Fisico en Protoboard

Las siguientes fotografias muestran el circuito LogicCalc construido fisicamente en protoboard, como complemento a la simulacion en Proteus.

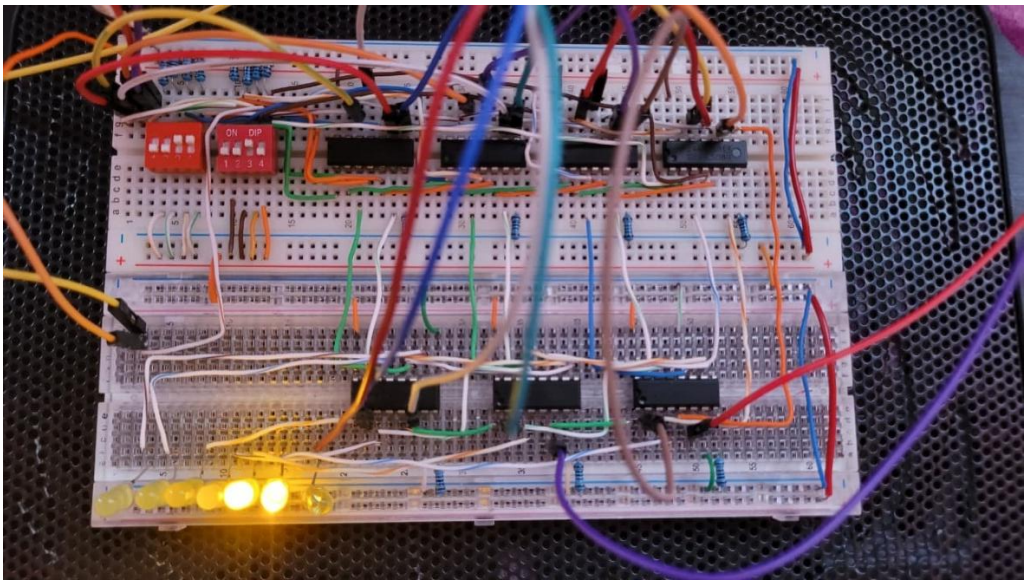
6.1 Controlador e indicadores

6.2 Sumador

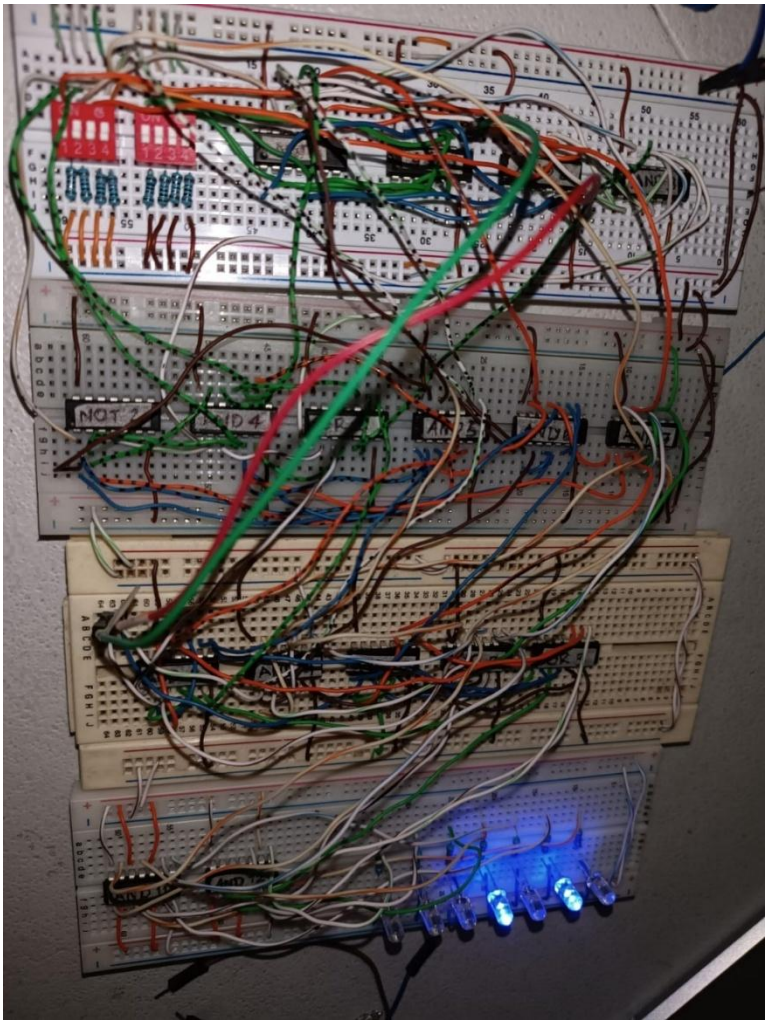
6.3 Restador



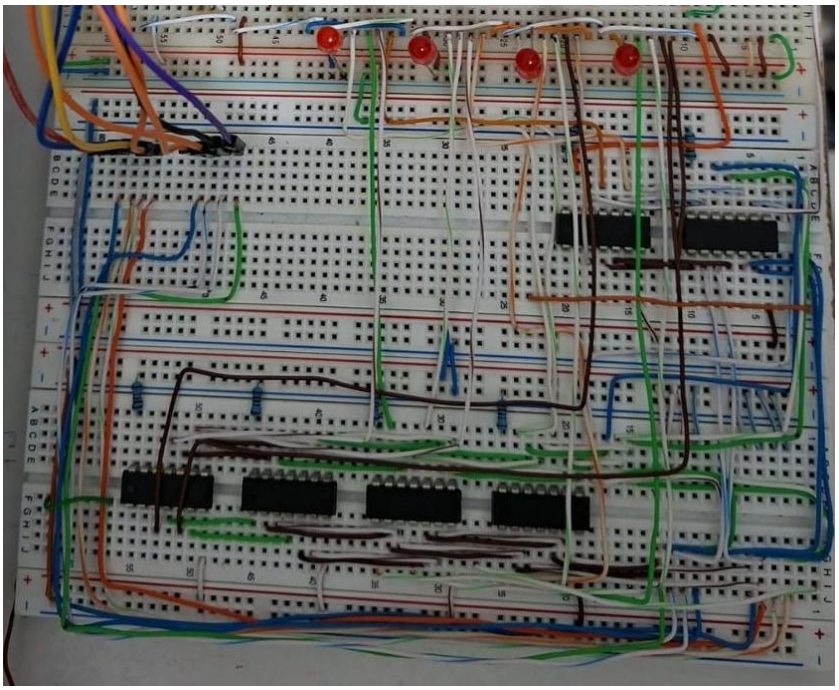
6.4 Multiplicador



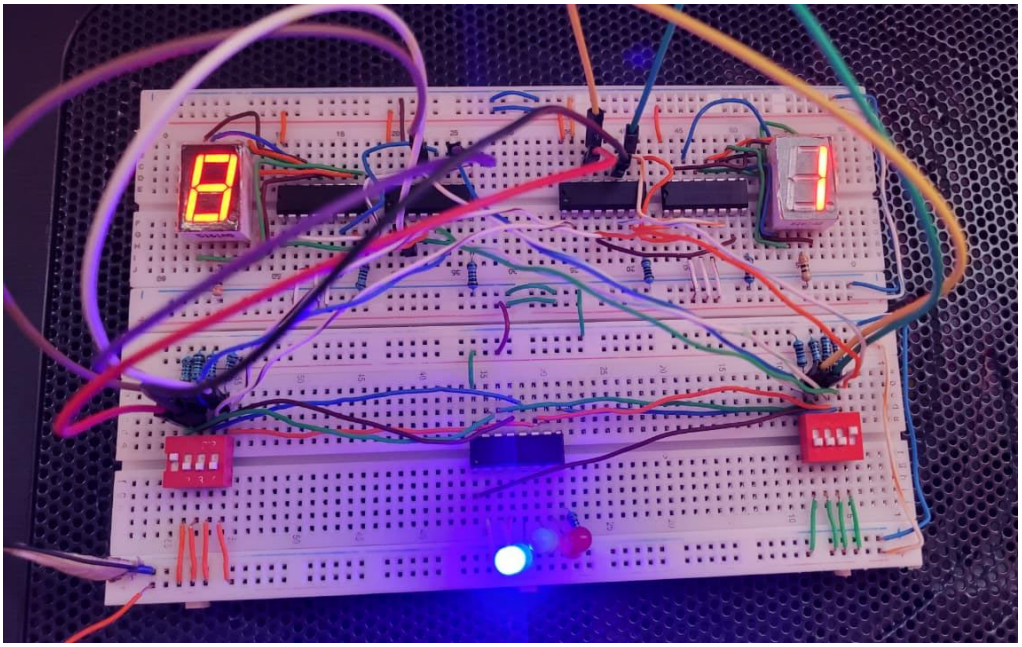
6.5 Potencia



6.6 Unidad logica



6.7 Unidad comparativa



7. Equipo Utilizado

Componente	Código	Cantidad
Integrado AND	**7408**	24
Integrado OR	**7432**	8
Integrado NOT	**7404**	6
Integrado XOR	**7486**	4
LEDs	**-**	23
DipSwitch 4 posiciones	**-**	8
DipSwitch 3 posiciones	**-**	1
Sumador	**7483**	8
Comparador	**7485**	6
Decoder BCD - 7 segmentos cátodo común	**7448**	4
Multiplexor	**74157**	2
Multiplexor	**74153**	7
Display 7 segmentos cátodo común	**-**	4
Resistencias (330, 1k)	**-**	-
Protoboard	**-**	10
Transistor	2N2222	1
Pushbutton de 4 pines		1
Cables y jumpers	**-**	-

8. Presupuesto

Componente	Cant.	P. Unit. (Q)	Subtotal (Q)
Integrado 7408	14	5	70
Integrado 7432	3	5	15
Integrado 7404	4	5	20
Integrado 7486	3	6	18
Integrado 7483	5	15	75
Integrado 7485	5	11	55
Integrado 74153	7	7	49
Integrado 74157	2	7	14
Integrado 7448	2	9	18
Integrado 7447	2	11	22
Displays 7 segmentos ánodo común	2	5	10
Displays 7 segmentos cátodo común	2	5	10
LEDs varios colores	10	1	10
Transistor 2N2222	___	___	
Resistencias (surtido)	18	0.5	9
Protoboard	1	40	40
Dip switch 4 posiciones	8	3.75	30
Jumpers	___	___	
Porta bateria de 9V tipo broche	1	3.25	3.25
Cable UTP (metro)	4	3	12
		TOTAL	480.25

9. Conclusiones

1

La construcción de la ALU LogicCalc permitió comprender de forma práctica cómo las operaciones fundamentales de un procesador, que parecen abstractas en teoría, se traducen en redes de compuertas lógicas físicas interconectadas. La implementación de la suma y la resta sin sumadores integrados, utilizando únicamente compuertas XOR, AND y OR con la técnica del complemento a dos, dejó en claro por qué este método es universalmente adoptado en la arquitectura de computadoras: simplifica el hardware al no requerir un restador dedicado. La visualización del acarreo propagándose a través de los bits fue especialmente reveladora para entender los límites de precisión numérica de un sistema de N bits.

2

El módulo del multiplicador evidenció que la multiplicación binaria, aunque conceptualmente sencilla (es solo suma repetida de productos parciales desplazados), requiere una cantidad significativa de hardware: 16 compuertas AND solo para los productos parciales, más los sumadores en cascada para la acumulación. Esto explica por qué en los procesadores reales los multiplicadores son los circuitos más grandes y lentos de la ALU, y por qué en arquitecturas embebidas muchas veces se implementan por software en lugar de hardware dedicado. El uso del 7483 como bloque de suma aceleró considerablemente la implementación y demostró el valor de los circuitos integrados combinacionales especializados.

3

La unidad comparativa demostró cómo un solo integrado, el 7485, encapsula una lógica que de otro modo requeriría decenas de compuertas para comparar dos números de 4 bits. La integración del comparador con los multiplexores 74157 para enrutar dinámicamente cuál operando va a cuál display fue un ejercicio muy completo de diseño de sistemas: no basta con obtener el resultado correcto, hay que también gestionarlo y presentarlo de forma adecuada. Asimismo, el decoder 7448 mostró cómo se abstrae la conversión BCD a 7 segmentos, una tarea que involucra siete funciones booleanas simultáneas implementadas internamente de manera muy eficiente.

4

El proyecto LogicCalc en su conjunto demostró que la teoría del álgebra de Boole no es un ejercicio matemático abstracto, sino la base directa de cada transistor que conmuta en un procesador. La experiencia de trazar una función booleana desde su tabla de verdad, pasarla por el mapa de Karnaugh para minimizarla, implementarla con compuertas físicas, simularla en Proteus y finalmente verla funcionar en el protoboard cierra el ciclo completo de comprensión en ingeniería. Los errores encontrados durante la construcción física (ruido, rebotes, problemas de fan-out TTL) añadieron una dimensión real que ninguna simulación puede reemplazar completamente.